

第四章 字符串与模式匹配

杨震 计算机学院

yangzhen@bupt.edu.cn

提 纲

BUPT

-  1 串的概念和基本操作
-  2 串的存储
-  2 串的模式匹配
-  3 KMP 算法
-  4 作业

4.1 串的概念和基本操作

BUPT

串（字符串）：是由零个或多个字符组成的有限序列。

记作： $s = 'a_1a_1...a_n'$ ($n \geq 0$)

串是特殊的线性表，数据元素是单个字符。

串长：串中字符的个数 n 。

子串和主串：串中任意个连续的字符组成的子序列称为该串的子串。包含子串的串称为主串。

串相等：两个串长度相等，且对应位置的字符都相等。

空串和空格串：空串不包含任何字符,表示为 ϕ ；
空格串由一个或多个空格组成，如 ‘ ’。

【顺序串的定义】

C语言：以\0结尾的字符数组。

C++：C语言中的数组继续有效，还提供std:string类。

教材书中默认下标从1开始，0存放串长度

```
typedef struct
{
    char data[Maxsize];\\存放串字符
    int length;\\存放串长
} SqString; \\顺序串类型
```

[串的基本操作]

(1)用串常量赋值	StrAssign(SqString &str,char cstr[])
用串变量赋值	StrCopy(SqString &s, SqString t)
(2)判等函数	StrCompare(SqString s, SqString t) (词典次序, $\geq \leq 0$)
(3)求长函数	StrLength(SqString s)
(4)连接函数	SqString Concat(SqString s, SqString t)
(5)求子串函数	SqString SubStr (SqString s, int i, int j)
(6)定位函数	Index(SqString s, SqString t, int i)
(7)置换函数	Replace(SqString &s, SqString sub, SqString t)
(8)插入操作	SqString InsStr(SqString s1,int i, SqString s2)
(9)删除操作	SqString StrDelete(SqString s,int i, int j)

****线性表的操作通常以“数据元素”为操作对象；串的操作主要以“串的整体”为操作对象。**

用串常量赋值

```
void StrAssign(SqString &str, char cstr[])
{
    int i;
    for(i=0; cstr[i] != '\0'; i++)
        str.data[i] = cstr[i];
    str.length = i;
}
```

用串变量赋值

```
void StrCopy(SqString &str, SqString t)
{
    int i;
    for(i=0; i<t.length; i++)
        str.data[i] = t.data[i];
    str.length = t.length;
}
```

求长函数

```
void StrLength(SqString s)
{
    return s.length;
}
```

连接函数

```
SqString Concat(SqString s, SqString t)
{
    SqString str; int i;
    str.length = s.length + t.length;
    for (i=0; i< s.length; i++)
        str.data[i] = s.data[i];
    for (i=0; i< t.length; i++)
        str.data[s.length+i] = t.data[i];
    return str;
}
```

求子串函数 从第*i*个字符开始连续*j*个字符

```
SqString SubStr (SqString s, int i, int j)
{
    SqString str; int k;
    str.length = 0;
    if( i <= 0 || i>s.length || j<0 || i+j-1>s.length )
        return str;
    for(k=i-1; k<i+j-1; i++) \\s的第i..i+j个复制给str
        str.data[k-i+1] = s.data[k];
    str.length = j;
    return str;
}
```

插入操作 *s2*插入*s1*的第*i*($1 \leq i \leq n$)个字符前

```
SqString InsStr(SqString s1,int i, SqString s2)
{
    SqString str; int j;
    str.length = 0;
    if( i <= 0 || i>s1.length+1 )
        return s1;
    for(j=0; j<i-1; j++) \\s1.data[0..i-2]复制给str
        str.data[j] = s1.data[j];
    for(j=0; j< s2.length; j++) \\s2全部->str
        str.data[i+j-1] = s2.data[j];
    for(j=i-1; j< s1.length; i++) \\s1剩余部分->str
        str.data[s2.length +j] = s1.data[j];
    str.length = s1.length+s2.length;
    return str;
}
```

设s='I am a student.'

t='OK!'

p='student'

q='nurse'

r='good '

(1) l=Concat (s, t)

l= 'I am a student. OK!'

(2) Replace(s, p , q);

s='I am a nurse.'

(3) l = InsStr(s, 8, r)

l='I am a good nurse.'

4.2 串的存储结构

BUPT

4.2.1 静态存储结构

1) 非紧缩格式 串空间+串长

```
#define m=18{用户定义串的最大长度}
struct {
    char data[1..m];
    int length;
};
```

{类型定义不用strlen时，应约定
标志串结尾的特殊字符}

特点：存储器采用字(16bit)编址结构时，
空间浪费较多。

1	A			
	B			
	O			
	O			
	K			
m				

2) 紧缩格式 串空间+串长

```
#define m=12{用户定义串的最大长度}
struct node{
    char data[1..m];
    int length;
};
```

特点：存储器采用字节编址结构时，节省空间且操作方便，否则分离同一存储单元字符需时间开销。

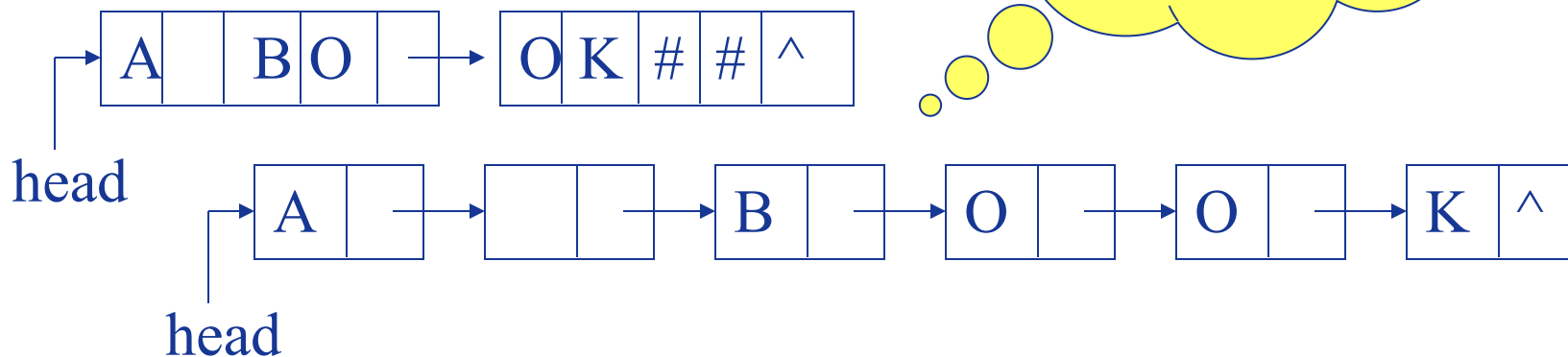
1	A		B	O
	O	K		
m				

4.2.2 动态存储结构

1) 块链存储方式 字符串块链表的头指针+串长

```
#define chunksize=4{用户定义的结点大小};
typedef struct chunk{
    char data[1..chunksize];
    struct chunk * next;
}
```

块(chunksize值)
大小的影响



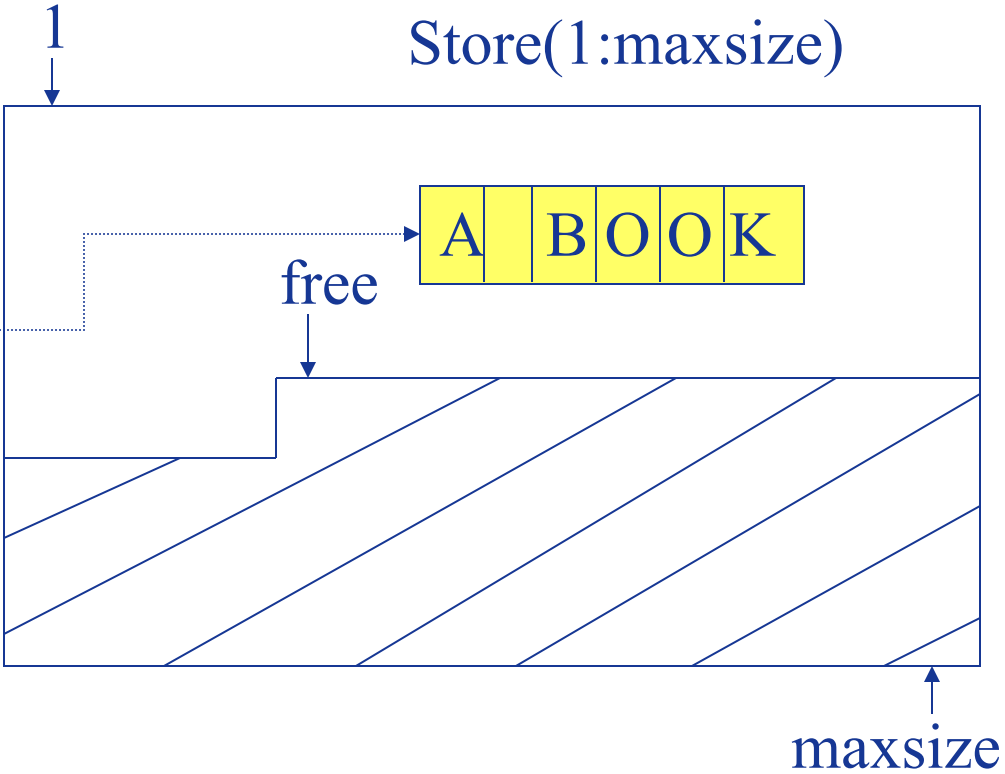
2) 堆结构存储方式

系统分配一个大容量连续存储空间作为串值的可利用空间。每个串使用时动态申请一组地址连续的存储单元。

符号表

a	8	1
b	6	9
c	13	22

串名 串长 起始地址
length stadr



堆结构存储的特点：

- ❖ 通过串名存储映象访问串值
- ❖ 动态分配存储空间，对串长没有限制
- ❖ 要使可利用空间的使用更加有效，回收废弃的旧串和申请新串空间需进行动态存储管理。



4.3 串的模式匹配

BUPT

[定义] 在主串中寻找子串（第一个字符 p_1 ）在主串中的位置 i ；匹配成功返回 i ($1 \leq i \leq n$)；匹配失败返回 0 。

在模式匹配中，子串又称为**模式串**，主串又称为**目标串**。

记模式串为： $p = p_1 p_2 \dots p_m$

记目标串为： $t = t_1 t_2 \dots t_n$

******[说明]：在模式匹配算法及描述中，假设字符串下标从1开始******

[示例]

目标 t ：“Beijing”

模式 p ：“jin”

匹配结果 = 4

若模式 p ：“xin”

匹配结果 = 0 // 失败

例：目标 $t = \text{'abcabcdefabcdmn'}$

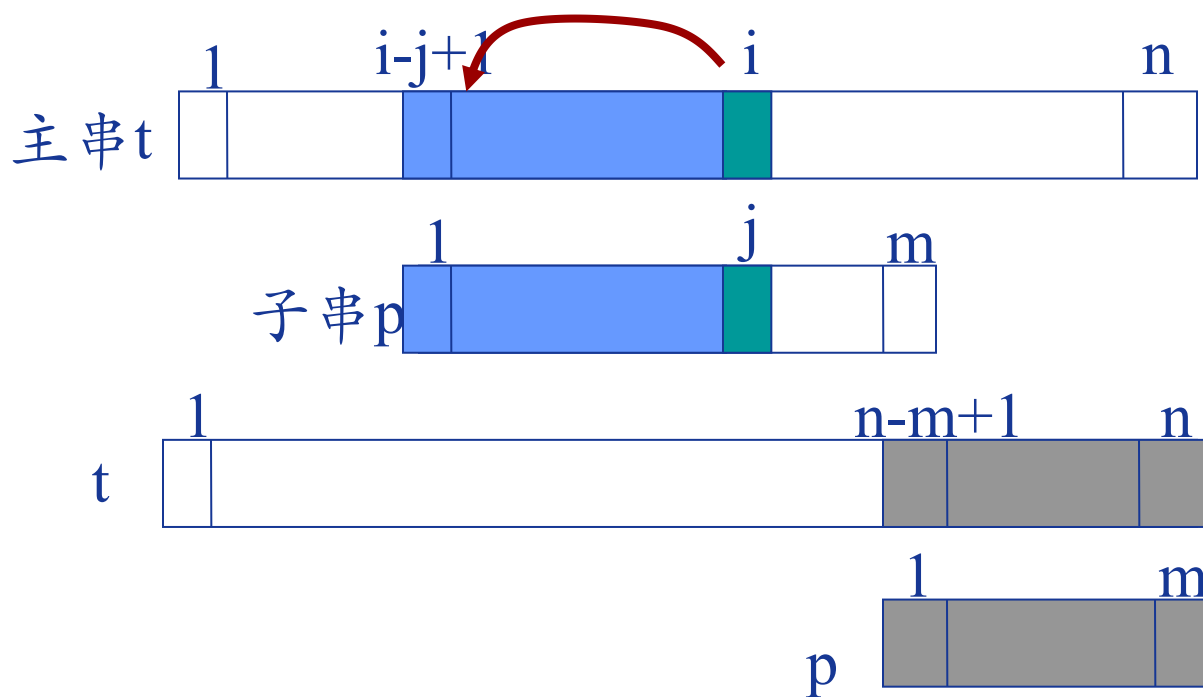
模式 $p = \text{'efapq'}$ 匹配失败，有效位移 = 0

模式 $p = \text{'bcd'}$ 匹配成功，有效位移 = 5

Brute-Force(BF)算法

BUPT

Brute-Force(BF)算法 又称 蛮力搜索算法：从目标串 t 第一个字符 t_1 开始与模式串 p 第一个字符 p_1 对比，若相等，则继续逐个比较后续字符，否则从目标串的第二个字符开始与模式串的第一个字符比较



Brute-Force(BF)算法描述

```

int index(char const *t, char const *p) // 蛮力搜索
{
    int i=1; j=1; // 假设字符数组的下标从1算起
    int n=strlen(t); m=strlen(p);
    while ((i <=n) && (j <=m) )
    {
        if (t[i]==p[j])
        {
            i=i+1;
            j=j+1;
        }
        else
        {
            i=i-j+2;
            j=1;
        }
    }
    if (j== m+1) \ 匹配成功
        return (i-m);
    else
        return(0);
}

```

【分析】

最坏情况字符比较次数
为: $(n-m+1)*m$

最坏情况下算法复杂度:
 $O(m*n)$, 具体取决于m和
n的关系

原因在于每趟重新比较
时, 目标串的检测指针i
要回退到 $i-j+2$ 。改进的
模式匹配算法可使目标
串的检测指针每趟不回
退。

【BF 算法匹配过程示例】

①

	1	2	3	4	5	6	7	8	9	10
主	a	b	a	b	a	b	a	b	c	a
子	a	b	a	b	c					

$\uparrow$ $i=5$ 修订为 $i=i-j+2=2$

$\downarrow$ $j=5$ 修订为 $j=1$

④

	1	2	3	4	5	6	7	8	9	10
主	a	b	a	b	a	b	a	b	c	a
子				a	b	a	b	c		

$\uparrow$ $i=4$ 修订为 $i=i-j+2=5$

$\downarrow$ $j=1$ 不变

②

	1	2	3	4	5	6	7	8	9	10
主	a	b	a	b	a	b	a	b	c	a
子	a	b	a	b	c					

$\uparrow$ $i=2$ 修订为 $i=i-j+2=3$

$\downarrow$ $j=1$ 不变

⑤

	1	2	3	4	5	6	7	8	9	10
主	a	b	a	b	a	b	a	b	c	a
子				a	b	a	b	c		

$\uparrow$ $i=10$

$\downarrow$ $j=6$ 成功

③

	1	2	3	4	5	6	7	8	9	10
主	a	b	a	b	a	b	a	b	c	a
子			a	b	a	b	c			

$\uparrow$ $i=7$ 修订为 $i=i-j+2=4$

$\uparrow$ $j=5$ 修订为 $j=1$

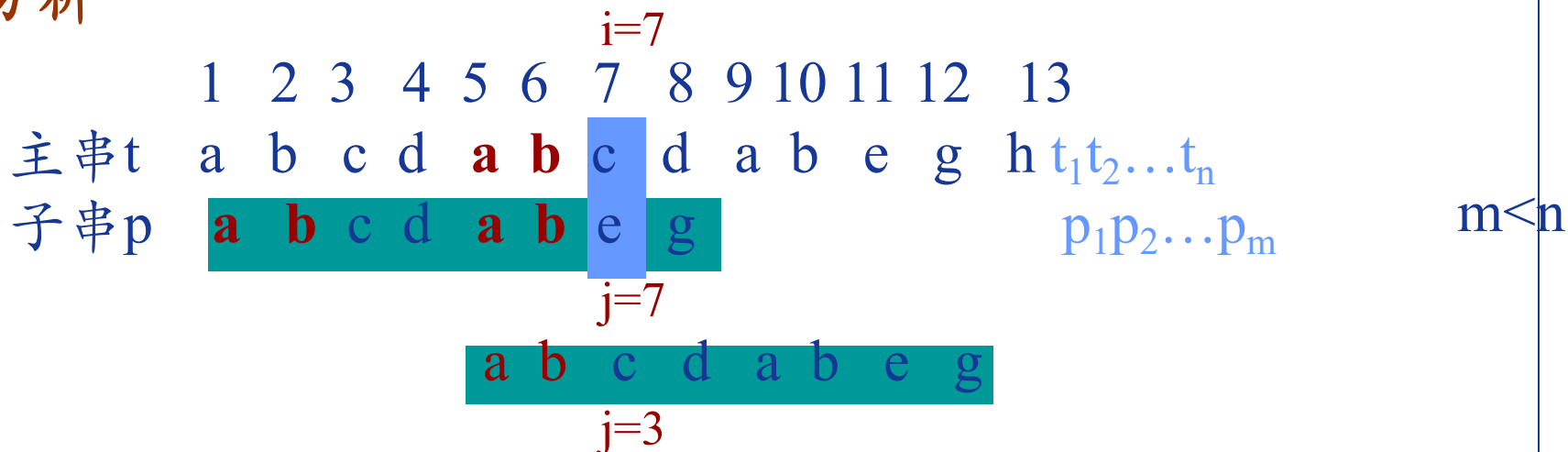
4.4 KMP 算法

BUPT

改进之处

在比较过程中，主串的下标 i 只增不减，不回溯，可使算法复杂度提高到 $O(m+n)$ 。

分析



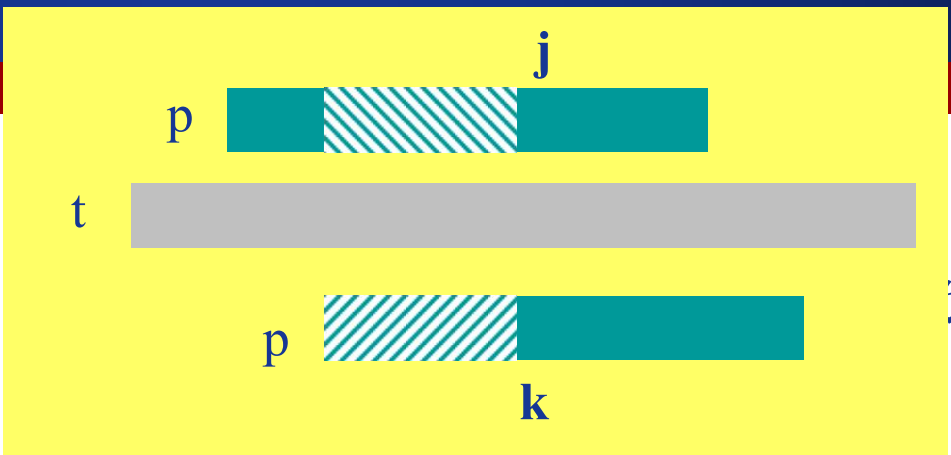
本趟比较失配点: $i=7, j=7$ (即 $t_7 \neq p_7$)

失配点处 $p_1 p_2 = t_5 t_6 = p_5 p_6 \implies p_1 \dots p_{k-1} = p_{j-k+1} \dots p_{j-1}$

下趟比较: i 不变, $j=k$ (此处即 3)

引入next数组

next[j]=k: k是当模式(子串
“失配”时, 在模式中
置。



next[j]=k

0 当 j=1 时 {代表下一趟比较 i=i+1, j=j+1=1}
max{k | 1<k<j 且 'p₁...p_{k-1}'='p_{j-k+1}...p_{j-1}'}
当此集合不空时
{下一趟比较 i 不变, j=k}

1 其它情况(即 j≠1 且上述集合为空)
{下一趟比较 i 不变, j=1}

j	1	2	3	4	5	6	7	8	9	10	11	12	13
	a	b	c	d	a	b	c	d	a	b	e	g	h
next[j]	0	1	1	1	1	2	3	4	5	6	7	1	1

j	1	2	3	4	5
	a	a	a	a	b
next[j]	0	1	2	3	4

KMP算法描述

```
Int KMP-index(char const *t, char const *p, int const *next)
{
    int i=1; j=1;
    int n = strlen(t); m = strlen(p);
    while((i<=n) && (j<=m))
    {
        if ((t[i]==p[j]) || j==0)
        {
            i++;
            j++;
        }
        else
            j=next[j];
    }
    if (j==m+1) \\ 匹配成功
        return (i-m);
    else
        return 0;
}
```

【KMP算法匹配过程示例】

模式串 $t = ababc$ $next[j] = 01123$

①

	1	2	3	4	5	6	7	8	9	10
主	a	b	a	b	a	b	a	b	c	a
子	a	b	a	b	c					

$i=5$ 不变

$j=5$ 修订为 $j=next[5]=3$

②

	1	2	3	4	5	6	7	8	9	10
主	a	b	a	b	a	b	a	b	c	a
子					a	b	c			

$i=7$ 不变

$j=5$ 修订为 $j=next[5]=3$

③

	1	2	3	4	5	6	7	8	9	10
主	a	b	a	b	a	b	a	b	c	a
子					a	b	a	b	c	

$i=10$

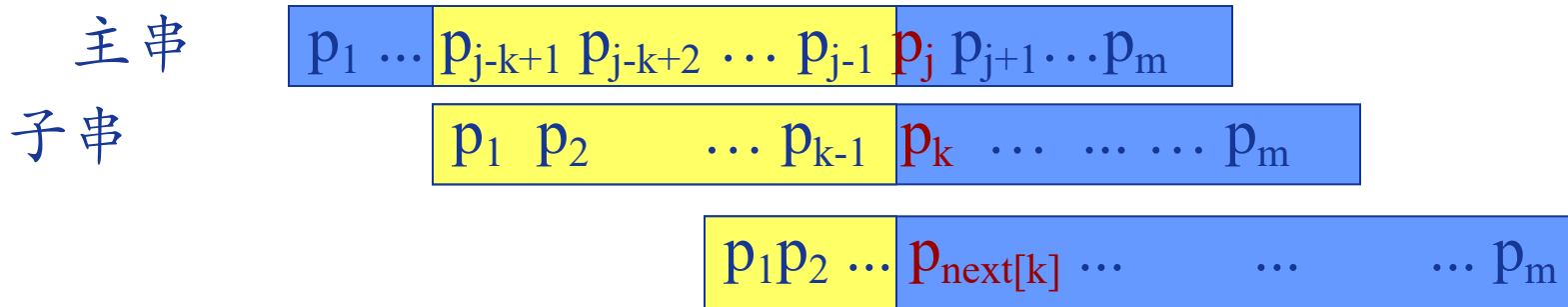
$j=6$ 成功

求next数组值

BUPT

算法思想：利用递推

即已知 $\text{next}[1]=0, \dots \text{next}[j]=k$; 求 $\text{next}[j+1]$



next求值算法描述

```
void get_next(int *next, char const *p) {
    int j=1; k=0; next[1] =0;
    while(j < strlen(p))
    {
        if ((k==0) || (p[k]==p[j]))
            next[++j]=++k;
        else
            k=next[k];
    }
}
```

修正next数组

BUPT

next数组的缺陷 i

T= a b c a b c a x a b c a b c a b b a c

P= a b c a b c a b b a c

next[8]=5

a b c a b c a b b a c

next[5]=2

a b c a b c a b b a c

next[2]=1

j a b c a b c a b b a c

存在 $p_8=p_5=p_2$, 因此当 $t_8 \neq p_8$ 时, t_8 与 p_5 、 p_2 的比较是无意义的。

改进方法 用nextval数组代替next数组

求出 p_j 处的k值; 若 $p_j=p_k$, 则 $\text{nextval}[j]=\text{nextval}[k]$

否则 $\text{nextval}[j]=k$

nextval求值算法描述

```
void get_nextval(int *nextval, char const *p)
{
    int j=1; k=0; nextval[1]=0;
    while (j < strlen(p))
    {
        if (k==0 || p[j]==p[k])
            if (p[++j] != p[++k])
                nextval[j]=k;
            else
                nextval[j]=nextval[k];
        else
            k=nextval[k];
    }
}
```

例	j	1	2	3	4	5	6	7	8	9	10	11		1	2	3	4	5
		a	b	c	a	b	c	a	b	b	a	c		a	a	a	a	b
	k	next[j]	0	1	1	1	2	3	4	5	6	1	2	0	1	2	3	4
		nextval[j]	0	1	1	0	1	1	0	1	6	0	2	0	0	0	0	4

改进的KMP算法描述

```
Int KMP-index(char const *t, char const *p, int const *nextval)
{
    int i=1; j=1;
    int n = strlen(t); m = strlen(p);
    while((i<=n) && (j<=m))
    {
        if ((t[i]==p[j]) || j==0)
        {
            i++;
            j++;
        }
        else
            j=nextval[j];
    }
    if (j==m+1)
        return (i-m);
    else
        return 0;
}
```

next数组修订前后匹配过程对比示例

模式串 $t=aaaab$ $next[j]=01234$ $nextval[j]=00004$

修订前 $next[j]=01234$

①

↑ $i=4$ 不变

1 2 3 4 5 6 7 8 9

主 a a a **b** a a a a b

子 a a a **a** b

↓ $j=4$ 修订为 $j=next[4]=3$

②

↑ $i=4$ 不变

1 2 3 4 5 6 7 8 9

主 a a a **b** a a a a b

子 a a **a** a b

↓ $j=3$ 修订为 $j=next[3]=2$

③

↑ $i=4$ 不变

1 2 3 4 5 6 7 8 9

主 a a a **b** a a a a b

子 a **a** a a b

↓ $j=2$ 修订为 $j=next[2]=1$

④

$i=4$ 修订为 $i=i+1=5$

1 2 3 4 5 6 7 8 9

主 a a a **b** a a a a b

子 **a** a a a b

$j=1$ 先 $j=next[1]=0$, 后 $j++$ 即 $j=1$

⑤

↑ $i=10$

1 2 3 4 5 6 7 8 9

主 a a a b **a** a a a b

子 a a a a **a** b

↓ $j=6$ 成功

修订后 nextval[j] = 00004

①

↑

i=4修订为i=i+1=5

1

2

3

4

5

6

7

8

9

主

a

a

a

b

a

a

a

a

b

子

a

a

a

a

b

↓

j=4 先 j=nextval[4] = 0, 后j=j+1=1

②

i=10

1

2

3

4

5

6

7

8

9

主

a

a

a

b

a

a

a

a

b

子

a

a

a

a

b

j=6 成功

KMP算法时间复杂度分析

通常，模式串长度 $m \ll$ 主串长度 n

求next(nextval)数组： $O(m)$

主串与模式串匹配： $O(n)$

整个匹配算法 $O(m+n)$

作业

BUPT

1. 分别求出下列模式串的next和nextval数组值

t1='aaab' t2='abcaabaa' t3='adabbadada'

2. 重复子串的含义是由一个或多个连续相等的字符组成的子串(不重叠);现有某个串用一维字符数组s存储, 长度为n, 设计算法求s的最长重复子串的长度.

int LongestString(char s[],int n) //函数返回所求长度